

w9L49 DQN Fitted Q-Iteration

Solving a Stationary regression problem

Fitted Q-iteration

$$\langle s_i, a_i, s'_i, r_i \rangle$$
$$\langle \Phi(s_i, a_i), \text{target}_i \rangle$$
$$\text{target}_i = r_i + \gamma \max_a \hat{Q}(s'_i, a)$$
$$\text{target}_i = r_i + \gamma \max_a \hat{Q}(s'_i, a)$$

For such tuples and give to function approximation

Requirement -

Should be able to run max

Static data should have enough representation - so across fi's sampled

Popular version:

Neural fitted Q - NFQ

Make the dataset and feed to NN

Problems you may face:

1. We should have enough samples
 - a. Index $f_i(x)$ with a - we are generating f_i target every time
2. Who gives f_i ? - generating f_i

Does Deep NN converge - no one knows! But it beats the world champion!

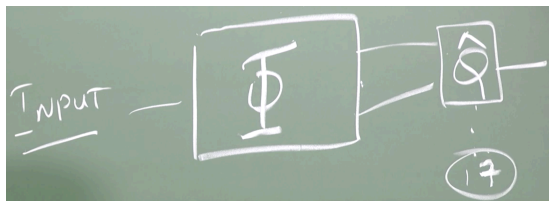
Don't Ask for Convergence!

DeepQN:

Addresses few problems

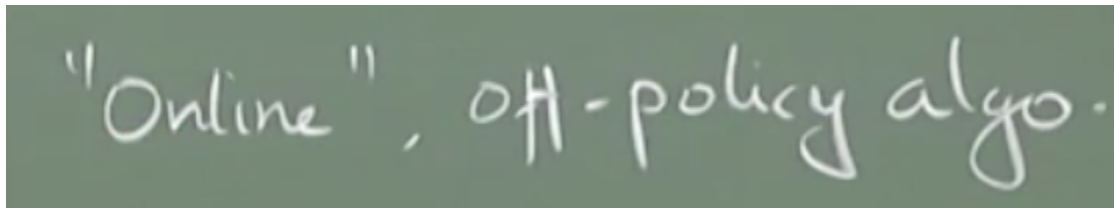
- Doesn't use offline - actual samples online (no need to worry about restrictive)
- It solves for f_i problem

Atari - 17 actions move in 8 direction, with fire another 8, no-op action, 18th fire and don't do anything (don't move joystick, press fire button)



Feed f_i , find next state, feed it for update - do in one shot

Don't do target generation first



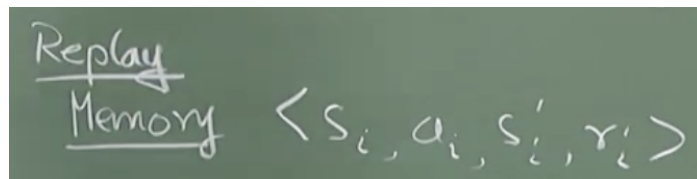
"Online", off-policy algo.

You will be updating along the trajectory

Problems in doing online - Q-learning

Bunch of things can wrong:

- Not using samples efficiently - just drawing one sampling, using and throwing away
 - Have a Replay Memory



Replay
Memory $\langle s_i, a_i, s'_i, r_i \rangle$

Efficient Re-Use

- Keep pulling this out of memory and replay it
- 1993 - Replay memory Yangi Lin - **remember these trajectories and apply**
- You replay Entire Trajectory
 - Sample he samples Trajectory Uniformly according to currently policy
 - **Doesn't sample Transitions**

Humans do replay - and take inspiration. Re-living your life's experience

Running simulation - building model and then sample

In Lin - take sample from real world - resample

- It is lot lighter
- On Atari - states are continuous, you are not fitting model
- You are NOT fitting the model
- Play one game and play again and again

What you converge to is certainty equivalence model

Org DQN had Memory of last 100 transitions

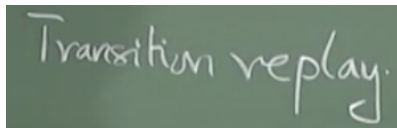
- In other variations - retain rare transitions, throw away trajectories
-

2nd Problem - similar update

- States are changing slowly, typically rewards will change slowly
- Successive states will look similar
- Updates will be strongly correlated - can lead to all kinds of problems with convergence

Pick one transition randomly from replay memory

- So don't have correlation
- Here it is sample
- Because it is Q-Learning, going to get from replay memory, doing max,



Biased sampling of input space

- If i decide to get left, will have display left of screen
- It will skew state space in left
- For Atari - inputs are pixels on screen
- Some state space get value function updated
- Unbalanced update in state space
 - All state on the right will get ignored
- You want whole state space to get updated
- It is problem with online learning

Takes care of issues

1. Inefficient use
2. Correlated updated
3. Skewed sampling

NN: Make hypothesis, if it works great. Otherwise change hypothesis

CNNs don't take care of all types of Symmetry

FI is Deep Learning

Org DQN

- two layer then one flat output layer
- recent three layer then one flat output layer

Input and output is q'

2) Which of the following is true about DQN?

- ☐ It may converge to non-optimal policy
- ☐ It is an off-policy technique
- ☐ It can be efficiently used for very large state spaces
- ☐ It can be efficiently used for continuous action spaces

No, the answer is incorrect.

Score: 0

Accepted Answers:

It may converge to non-optimal policy

It is an off-policy technique

It can be efficiently used for very large state spaces

Question1: DQN is Off-line - NOT online

Question 2:

6) Why is an experience replay buffer used instead of direct updates to network in DQN?

- ☐ Random sampling from experience replay buffer breaks correlations among transitions
- ☐ If not, the off-policy nature of Q-Learning is lost
- ☐ It guarantees convergence to the optimal policy
- ☐ None of the above

No, the answer is incorrect.

Score: 0

Accepted Answers:

Random sampling from experience replay buffer breaks correlations among transitions

w9L50 Policy Gradient Approach

1st Adv:

- Compact Repr.

Don't try to find value function

- What is it good idea to not find value function
- Value function is complex - distribution of value fn over states than policy
 - Example - inventory control
 - Cross product to things are available

Policy can be represented by by fewer parameters

2nd Adv:

- "Better" convergence behavior.

- Value fn - don't have much intuition - how it will work
 - It may diverge, no guarantee of methods
- POLICY - At Least converge to local optimum

In practice value function works

All Large Scale problems - involve VALUE function

- XOR counter example - value fn may not converge
- Bandit doesn't need DNN

Policy π depends on θ - π parameterised by θ

- $\pi(\cdot, \theta)$

Move in direction of gradient - as maximising

- Will reach local max/min
- Gradient estimate should be unbiased
- Small step size

$$\eta(\theta) = \sum_a q^*(a) \pi(a; \theta)$$
$$\nabla \hat{\eta}(\theta) = \frac{1}{N} \sum_{i=1}^N r_i \cdot \frac{\nabla_{\theta} \pi(a_i; \theta)}{\pi(a_i; \theta)}$$

Want to do it for FULL RL problem:

- What is the most straight forward way to extend it

Policy: $\pi(\cdot, \cdot, \cdot; \theta)$

Q function was average fn of rewards

For full - average of returns - i should be collecting trajectories,
Samples will be of following form

$$\pi(\cdot, \cdot, \cdot; \theta)$$
$$\langle s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_T \rangle$$

$$\nabla \hat{q}(\theta) = \frac{1}{N} \sum_{i=1}^N G_i(s_0) \cdot \frac{\nabla p_i(s_0; \theta)}{p_i(s_0; \theta)}$$

G_i - return on i-th sample, $p_i(s_i, \theta)$ p of seeing sample from s_0

$$G_i(s_0) - \text{Return of } i^{\text{th}} \text{ sample, starting from } s_0$$
$$p_i(s_0; \theta) - \text{prob of seeing } i^{\text{th}} \text{ sample, starting from } s_0$$

s_0, a_0, s_1, a_1

Prob of G_i is 1

Likelihood of taking G_i under policy θ

Can convert product term into sum term - it is derivative of log

$$\frac{\nabla p_i(s_0; \theta)}{p_i(s_0; \theta)} = \nabla \ln p_i(s_0; \theta)$$

π is taking s_0, a_0, s_1, a_1 , product is returned as

16min: Derivative will cancel out - so it will be zero?

It reduces to prob of taking s_0, a_0, s_1, a_1

$$\frac{\nabla p_i(s_0; \theta)}{p_i(s_0; \theta)} = \nabla \ln p_i(s_0; \theta)$$

$$= \sum_{j=0}^{T-1} \frac{\nabla \pi(s_j, a_j; \theta)}{\pi(s_j, a_j; \theta)}$$

Write pi as product, take ln, cancel out, left out terms are written
It is likelihood ratio

Once I estimate gradient, i can do increment, online update

Performance measure (eta) - depends on theta alone

- No value fn
- What have i done to give single value for entire policy

What I have done here
s0 does not have ' on it

Unique start state:

In many in cases - unique start state

- Start with same start state
- In many cases it is acceptable

You have set of start state

- Without action sample one state

Start State $G_i(s_0)$ - Return of i^{th} sample, starting from s_0

s_0 is unique start state

Assumption for single value of policy:

- Average reward RL - what does it tell you - take average over N steps
- Any initial variation will wipe out as you will take average

value of policy denoted by π - MDP satisfies

Instead of $G_i(s_0)$ use ρ_{π}

You are not running till end of infinity

$$\rho_{\pi}(s) = \sqrt{\pi} \quad \forall s$$

$$= \mathbb{E} \left[\lim_{N \rightarrow \infty} \frac{R_1 + R_2 + \dots + R_N}{N} \right]$$

Average Reward RL - one full lecture

- Certain convergence results easier to prove
- Don't have to evaluate gamma
 - for some problems - naturally defined
 - Try artificially define what it should be - meta parameter

Function approximation

- Well behaved for Average reward, then discounted function
- Many advantages are theoretical
- When you change discount factor slightly - results different
- Numerically - average reward hard to work

Easier to show result when you have policy gradient

If all trajectories are of finite length

- Always assume N tends to infinity -
- If recurrent state absorbs - you do infinite transition
- Under these policy, you keep visiting
- Average rewards is important

MC Policy Gradient

-

Run policy gradient on Puzzle world!

What will be the biggest problem with MC Policy Gradient:

- trajectories are very long
 - Significant variance in sample is high

You run something for ever

What can you do to reduce the variance

Is it possible to truncate trajectories

- I will only do SHORT term
- If using average reward - will have good estimate
- Roughly same
- Need to have some way of stopping

Whenever i reenter the recurrence state - will stop the trajectory

You can designate multiple states where you stop

Discounting

33min: Another method for reducing sample

- From where variance coming
- It is coming from G
- Do i know which will have low variance - value functions
 - If you plug some function derived from value functions

Actor Critic

- Actor - π , actor - θ
- Critic evaluate fn - evaluate the actor
- Critic comes from any of favourite value fn - use some fn approx
- That will feed in Actor fn approximation

How do I take out return and replace with value function

- Relationship between Actor and Critic
-

8) Choose the correct statement for Policy Gradient Theorem for average reward formulation:

☐

$$\frac{\partial \rho(\pi)}{\partial \theta} = \sum_s d^\pi(s) \sum_a \frac{\partial \pi(s,a)}{\partial \theta}$$

☐

$$\frac{\partial \rho(\pi)}{\partial \theta} = \sum_s v^\pi(s) \sum_a \frac{\partial \pi(s,a)}{\partial \theta} q^\pi(s,a)$$

☐

$$\frac{\partial \rho(\pi)}{\partial \theta} = \sum_s d^\pi(s) \sum_a \frac{\partial \pi(s,a)}{\partial \theta} q^\pi(s,a)$$

☐

None of the above

No, the answer is incorrect.

Score: 0

Accepted Answers:

$$\frac{\partial \rho(\pi)}{\partial \theta} = \sum_s d^\pi(s) \sum_a \frac{\partial \pi(s,a)}{\partial \theta} q^\pi(s,a)$$

Question 6:

w9L51 Actor-Critic REINFORCE

REINFORCE:

- Update rule:

$$\Delta \theta = \alpha_n (r_n - b_n) \frac{\partial \ln \pi(a_n, \theta_n)}{\partial \theta}$$

b_n - reinforce baseline - average of all previous rewards

If $r_n > b_n \rightarrow$ it is a good action

If $r_n < b_n \rightarrow$ it is a bad action

$$\Delta \theta = \alpha_n (r_n - \underbrace{b_n}_{(G_n(s_n) - b_n(s_n))}) \frac{\partial \ln \pi(a_n, \theta_n)}{\partial \theta}$$

How do i extend it to full RL $\rightarrow$ make it function of s . R_n will become G_t

$$\underbrace{(G_n(s_n) - b_n(s_n))}_{\text{look at } b_n(s_n)}$$

look at $b_n(s_n)$

- average of all returns you get starting from s_n

- it is definition of value function

$$(G_n(s_n) - \underbrace{V_n(s_n)})$$

Still solving for the parameter, in evaluation using value function

Policy as well as value function

$$\Delta \theta = \alpha_n (r_n - \underbrace{b_n}_{(G_n(s_n) - \underbrace{V_n(s_n)}_{\text{Critic}})}) \frac{\partial \ln \pi(a_n, \theta_n)}{\partial \theta}$$

Actor

Basic idea of policy parameterisation, gives rise to Actor-Critic

One way of thinking about what Actor critic is
Can replace G_n with

$$\Delta \theta = \alpha_n (r_n - (b_n)) \frac{\partial \ln \pi(a_n, \theta_n)}{\partial \theta}$$

$\downarrow$
 Actor $(G_n(s_n) - V_n(s_n))$ Critic
 $\downarrow$
 $(R_{n+1} + \gamma V_n(s_{n+1}) - V_n(s_n))$
 TD error, δ_n

This term $(r_n - b_n)$ has become magically TD error

Some kind of lookup table - if action is taken =1, otherwise 0

We can write like TD update rule

$$\alpha_n \delta_n \phi(l_n)$$

use linear - updation for policy parameter

What will be the update for value parameter?

- TD update

What is the difference between this and TD update rule

Something big is missing here

Remember **Actor**, if I took the action, I will change the parameter, otherwise won't

For **Critic** state case, regardless of action I take, I will update value function

- Everytime I visit the state

Actor and critic will be changed by the same amount $\alpha_n \delta_n$

$$\Delta \theta = \alpha_n \delta_n \phi(l_n, s_n)$$

Choose parameters appropriately

This is basic Actor-Critic method

- In book it is presented before function approximation
- NOT after function approximation
- Same text from 1st edition

1st - we are generalising from RL baseline

- Then get into Actor-Critic

2nd Motivation for Actor-Critic

- Policy gradient has lot of variance
- We want to reduce variance
- instead of returns bring in value function in the setting

10) Using similar parameterizations to represent policies, Monte Carlo policy gradient methods would converge to a locally optimal policy faster than Actor critic method in which approximation in critic is compatible with actor parameterization?

- ☐ True
☐ False

No, the answer is incorrect.

Score: 0

Accepted Answers:

False

Question 8

10) Using similar parameterizations to represent policies, Monte Carlo policy gradient methods would converge to a locally optimal policy faster than Actor critic method in which approximation in critic is compatible with actor parameterization?

- ☐ True
☐ False

No, the answer is incorrect.

Score: 0

Accepted Answers:

False

Question 4

5. In using policy gradient methods, if we make use of the average reward formulation rather than the discounted reward formulation, then is it necessary to consider, for problems that do not have a unique start state, a designated start state, s_0 ?

- (a) no
(b) yes

Sol. (a)

We used the concept of a designated start state to allow a single value that can be assigned to a policy for the purpose of evaluation. The same result is obtained when using the average reward formulation, i.e, by using the average reward formulation we can compare policies according to their long term expected reward per step, $\rho(\pi)$, where

$$\rho(\pi) = \lim_{n \rightarrow \infty} \frac{1}{N} E\{r_1 + r_2 + r_3 + \dots + r_N | \pi\}$$

Question 9

2. In the Deep Q-learning method, is ϵ -greedy (or other equivalent techniques) required to ensure exploration, or is this taken care of by the randomisation provided by experience replay?

- (a) no
(b) yes

Sol. (b)

Some technique to ensure exploration is still required. As with the original Q-learning algorithm, if we only consider transitions generated by the action value function (which is essentially what we'll get with experience replay without any exploration), a large part of the state space will likely remain unexplored.

Question 10

4) **Assertion:** Actor-critic updates have lesser variance than REINFORCE updates

Reason: Actor-critic methods use TD target instead of G_t

- ☐ Both Assertion and Reason are true, and Reason is correct explanation for Assertion
☐ Both Assertion and Reason are true, but Reason is not correct explanation for assertion
☐ Assertion is true, Reason is false
☐ Both Assertion and Reason are false

No, the answer is incorrect.

Score: 0

Accepted Answers:

Both Assertion and Reason are true, and Reason is correct explanation for Assertion

Question 5

4. Suppose we are using a policy gradient method to solve a reinforcement learning problem. Assuming that the policy returned by the method is not optimal, which among the following are plausible reasons for such an outcome?

- (a) the search procedure converged to a locally optimal policy
- (b) the search procedure was terminated before it could reach the optimal policy
- (c) the sample trajectories arising in the problem were very long
- (d) the optimal policy could not be represented by the parametrisation used to represent the policy

Sol. (a), (b), (d)

Option (c) may result in an increase in the time it takes to converge to a policy, but does not necessarily affect the optimality of the policy obtained.

Question 7

1) **Statement:** DQN is implemented with current and target network.

Reason: Using target network helps in avoiding chasing a non-stationary target

- ☐ Both Assertion and Reason are true, and Reason is correct explanation for Assertion
- ☐ Both Assertion and Reason are true, but Reason is not correct explanation for assertion
- ☐ Assertion is true, Reason is false
- ☐ Both Assertion and Reason are false

No, the answer is incorrect.

Score: 0

Accepted Answers:

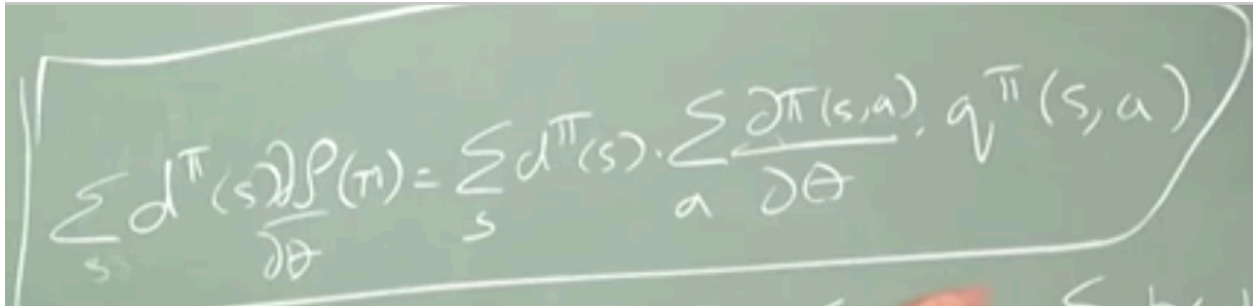
Both Assertion and Reason are true, and Reason is correct explanation for Assertion

Question 3

ToDo: w9L52 Proof of Theorem I

NOT There

w9L53 Policy Gradient with Function Approximation


$$\sum_s d^\pi(s) \frac{\partial J(\pi)}{\partial \theta} = \sum_s d^\pi(s) \cdot \sum_a \frac{\partial \pi(s,a)}{\partial \theta} \cdot q^\pi(s,a)$$

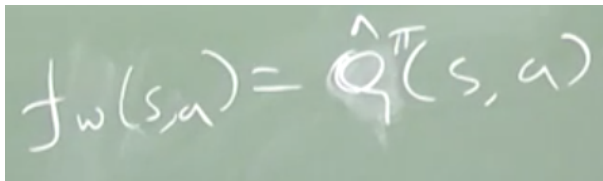
Estimating gradient - exact estimation of gradient

If you follow the policy gradient it will converge

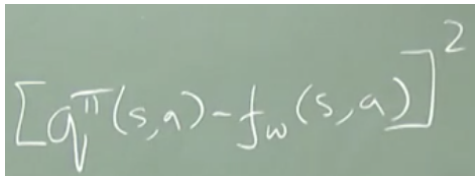
No way you can get q-pi

If approximate-pi -> some para w rep q fn, some para theta represent policy

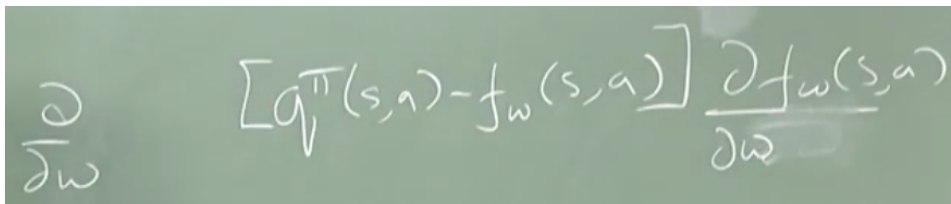
When can i guarantee some kind of convergence - some Q^π


$$f_w(s,a) = \hat{Q}^\pi(s,a)$$

Look at the error - squared error


$$[q^\pi(s,a) - f_w(s,a)]^2$$

Changes should be proportional to the gradient of this function


$$\frac{\partial}{\partial w} [q^\pi(s,a) - f_w(s,a)] \frac{\partial f_w(s,a)}{\partial w}$$

Go in opposite direction so minus will go away

How often i am likely to make this change

How often I will change s(a)

If run for long time

$$d^{\pi}(s) \pi(s, a) [q_v^{\pi}(s, a) - f_w(s, a)] \frac{\partial f_w(s, a)}{\partial \omega}$$

Total change - goes to zero.

$$\sum_s d^{\pi}(s) \sum_a \pi(s, a) [q_v^{\pi}(s, a) - f_w(s, a)] \frac{\partial f_w(s, a)}{\partial \omega} = 0$$

Let us assume it has converged - value fn has converged

Thm If f_w satisfies ①, and

$$\frac{\partial f_w(s, a)}{\partial \omega} = \frac{\partial \pi(s, a)}{\partial \theta} \cdot \frac{1}{\pi(s, a)}$$

then $\frac{\partial J}{\partial \theta} = \sum_s d^{\pi}(s) \cdot \sum_a \frac{\partial \pi(s, a)}{\partial \theta} \cdot f_w(s, a)$

consistent condition

Doing fn, approximation, it converges to zero

If it is consistent wrt para chose for pi

Get similar condition - but approximation function put in

What it really tells:

- You need to get relative ordering correct
- Fw could be very wrong

What does the convergence condition tell?

$$\sum_s d^{\pi}(s) \sum_a \pi(s, a) [q_v^{\pi}(s, a) - f_w(s, a)] \frac{\partial f_w(s, a)}{\partial \omega} = 0$$

Let us do special case, then it will be clear

$$\pi(s, a) = \frac{e^{\theta^T \phi_{sa}}}{\sum_b e^{\theta^T \phi_{sb}}}$$

Softmax kind of function

Represents state, action pair, s,a

What does the condition mean? Take derivative of softmax

$$\frac{\partial f_w(s, a)}{\partial w} = \frac{\partial \pi(s, a)}{\partial \theta} \cdot \frac{1}{\pi(s, a)}$$

take derivative

You will get - in the direction policy is varying - error = 0

Projecting error q-pi

My value fn approximation is sufficient to rep any variation i will have in this policy

- Prob of picking vale fun in that direction is zero
- Orthogonality is what we are looking

Value function has be nuance

In the direction of gradient

$$[q_V^\pi(s, a) - f_w(s, a)] \frac{\partial f_w(s, a)}{\partial w} = 0 \quad \text{--- (1)}$$

error will be zero

Project this error, expected value (wrt dpi(s) should be zero

Consistency condition

$$\frac{\partial f_w(s, a)}{\partial w} = \phi_{sa} - \sum_b \pi(s, b) \cdot \phi_{sb}$$

It is the update for softmax

Easy way of achieving it

$$f_w(s,a) = w^T \left[\phi_{sa} - \sum_b \pi(s,b) \phi_{sb} \right]$$

Make it linear in that quantity:

- This is the ONLY way of doing it
- Linear way of doing - f_i - is rep for policy
- Value is some combination of f_i

For a given state, what will be the mean of feature vector

$$\frac{1}{|A|} \sum_a f_w(s,a) \cdot \pi(s,a) = 0$$

Essentially, taken same policy para, wrt $\pi(s,a)$ made it zero

Made the parameterization for the value fn

That is the relationship between policy and value parameterisation

It guarantees, i can plug value fn

Instead of q_w plug f_w

$$\frac{\partial J}{\partial \theta} = \sum_s d^{\pi(k)} \cdot \sum_a \frac{\partial \pi(s,a)}{\partial \theta} \cdot f_w(s,a)$$

In the direction of $d\pi/d\theta \rightarrow$ the expected error will be zero

Simple thing in hind side

Very powerful result

- Actually use value fn - under consistency condition
- Guaranteed convergence

For any kind of arbitrary differentiable (assume gradient exists) parameterisation

Consistency condition will be satisfied

- Only when it is linear